

Explicar las decisiones de diseño tomadas.

o ¿Cómo manejaron la herencia y el polimorfismo en su diseño? ¿Qué ventajas les brindó?

Se diseñó una clase base llamada Usuario, de la cual heredan Estudiante e Instructor, lo que permitió evitar la duplicación de atributos comunes como nombre y carnet, y garantizó que en el futuro sea sencillo agregar nuevos tipos de usuarios heredando estas propiedades básicas. Asimismo, se aplicó polimorfismo en la jerarquía de Actividad → Tarea y Evaluación, ya que ambas comparten atributos generales (título y fecha), pero redefinen el método `mostrar_info()` para personalizar la salida según el tipo de actividad. Esta decisión brindó flexibilidad y promovió la reutilización de código, evitando estructuras repetitivas y haciendo el sistema más escalable y mantenible.

o ¿Qué estructuras de datos fueron más útiles y por qué?

En el desarrollo del sistema se utilizaron principalmente diccionarios y listas, ya que ofrecieron una organización clara y eficiente de la información. Los diccionarios (`dict`) resultaron fundamentales para almacenar usuarios (`{carnet: Usuario}`) y cursos (`{código: Curso}`), debido a que permiten búsquedas rápidas a través de claves únicas. Además, fueron empleados en las calificaciones de los estudiantes (`{curso: {evaluacion: {"nota": valor, "extra": info}}}`), lo que facilitó una estructura jerárquica ordenada y fácil de consultar. Por su parte, las listas (`list`) se usaron para registrar las actividades de cada curso en orden cronológico, garantizando un acceso secuencial sencillo. Gracias a estas estructuras de datos, se logró eficiencia en las búsquedas y una organización intuitiva que redujo significativamente la complejidad de las consultas dentro del sistema.

o ¿Qué errores comunes anticiparon y cómo los controlaron en el código?

En el diseño del sistema se anticiparon varios errores comunes y se implementaron mecanismos para controlarlos. Por ejemplo, se validó que las calificaciones fueran siempre valores numéricos (enteros o flotantes), se evitó la duplicidad de usuarios y cursos mediante la verificación de carnets y códigos ya registrados, y se comprobó que el instructor existiera antes de asignarle un curso. Además, en los casos en que un estudiante no tuviera calificaciones registradas, se devolvía un valor por defecto (0 o None) para prevenir errores de división entre cero. Finalmente, se integró el uso de bloques `try/except` para capturar

excepciones inesperadas y mostrar mensajes claros al usuario, garantizando la continuidad y estabilidad del programa sin que se interrumpiera su ejecución.

o ¿Cómo organizaron su trabajo usando Git y GitHub?

Como inicio del proyecto se creó un repositorio en GitHub y lo clonamos

Trabajamos en 2 ramas separadas, evitando modificar directamente la rama principal.

Realizamos commits frecuentes para documentar el avance paso a paso.

Sincronizamos los cambios con git add antes de trabajar y con git push para subirlos al repositorio remoto. Específicamente se clasificó en 2 personas que se encargaron del código en con las clases y funciones, la tercera persona realizó el menú principal para llevar un orden.

o ¿Qué mejoras harían si tuvieran más tiempo?

Se podría implementar la interfaz gráfica para facilitar el manejo del usuario, así como realizar la lógica más flexible. Se podría agregar funciones para que el usuario pueda ver por ejemplo su cuenta corriente. También sería

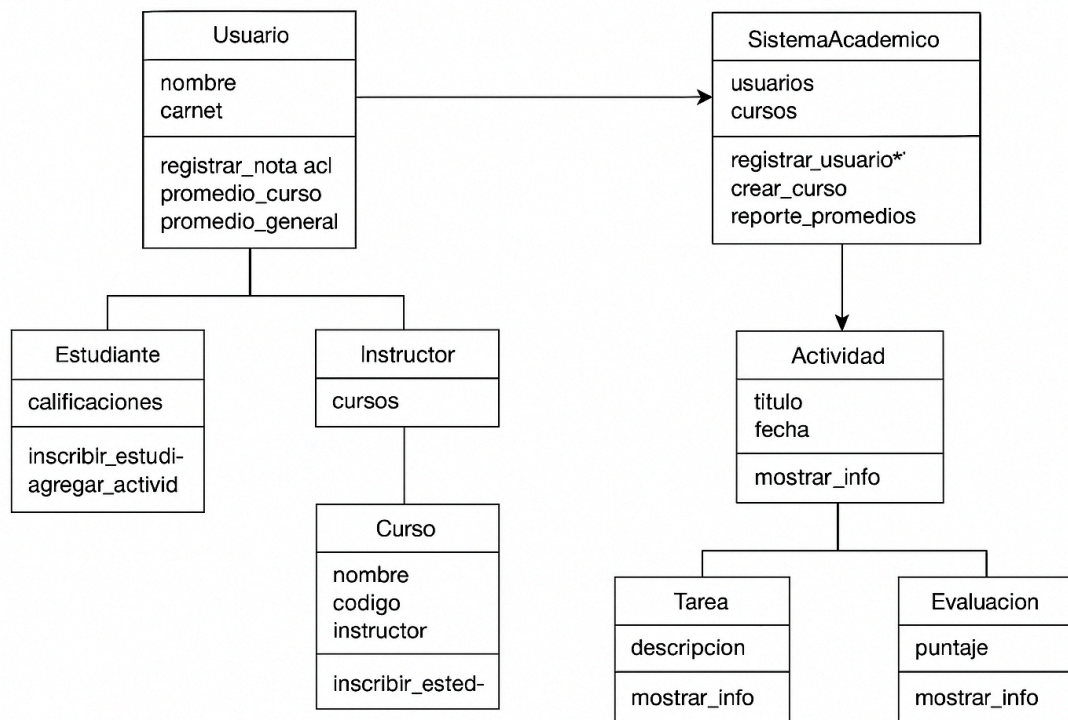
una buena idea implementar un apartado de anuncios para notificar a la comunidad educativa sobre actividades o información sobre los cursos.

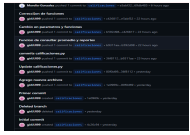
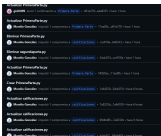
▪ Reflejar las dificultades encontradas y cómo las resolvieron.

Al inicio surgió la dificultad de estructurar correctamente las relaciones entre las clases, ya que había confusión sobre qué responsabilidades debían tener Curso, Estudiante e Instructor; esto se resolvió aplicando el principio de responsabilidad única, definiendo con claridad el rol de cada clase. Para evitar duplicar código se implementa la herencia en las clases Usuario y Actividad, centralizando atributos comunes y logrando mayor reutilización. Otro reto fue controlar los errores silenciosos, ya que sin validaciones el sistema aceptaba datos inválidos, lo cual se corrigió mediante verificaciones con condiciones claras y el uso de bloques try/except para capturar excepciones. Finalmente, la integración de todo en el SistemaAcadémico representó un desafío al conectar cursos, estudiantes y actividades, pero se resolvió utilizando estructuras jerárquicas basadas en diccionarios y listas, lo que facilitó el acceso y la organización de la información.

▪ Mostrar diagramas y pseudocódigo.

Repositorio GitHub con historial y ramas



Integrante	Carnet	Tarea Asignada	Evidencia en GitHub	Fecha de Entrega	Validacion docente
Keila Belinda Puac Gomez	1660625	Registro e Inicio de Sesión de Usuario/ Clase y funciones de Curso		18-09-25	
Zoe Andrea López Miche	1591125	Registrar evaluaciones por estudiante y evaluación consultar cursos, estudiantes y evaluaciones Generar reportes por promedio.		18-09-25	,
Morelia Gonzalez Ixcot	1536325	Documentación y Pseudocódigo y menú de python		18-09-20 25	

